

# SE-1 – Übungsblatt Nr. 1: Antworten

## Aufgabe 1.1 – Factory und Kommunikationsverbindung

**Wie wird die Verbindung zwischen Client und GermanTranslator aufgebaut, und wo liegt die Factory?** Die Objekterzeugung übernimmt die Klasse TranslatorFactory mit der statischen Methode createGT(). Sie erzeugt ein GermanTranslator-Objekt, setzt das aktuelle Datum (Format dd.MM.yyyy) und gibt das Objekt als Translator zurück. Der Client ruft nur TranslatorFactory.createGT() auf und arbeitet danach ausschließlich mit dem Interface Translator. Er verwendet selbst kein new. Die Factory liegt im Package control, also neben Translator und GermanTranslator. Die Objekterzeugung gehört zur Anwendungslogik und nicht zur View. Außerdem soll laut Kommentar nur systemintern (durch die Factory) und nicht von der View das Datum gesetzt werden.

**Welches Entwurfsmuster wird verwendet, und welchen Nutzen hat es?** Verwendet wird das Erzeugungsmuster **Factory Method** (Fabrikmethode, GoF). Es hat folgenden Nutzen: - **Lose Kopplung:** Der Client hängt nur vom Interface ab, nicht von der konkreten Klasse. - **Austauschbarkeit und Erweiterbarkeit:** Eine andere Implementierung (z. B. ein EnglishTranslator) lässt sich einbinden, indem man nur die Factory ändert. Der Client bleibt unverändert (Open-Closed-Prinzip). - **Zentrale Erzeugungslogik:** Initialisierungen wie das Setzen des Datums stehen an genau einer Stelle und können nicht vergessen werden. - **Bessere Testbarkeit und Wartbarkeit**, weil die Abhängigkeiten klar getrennt sind.

**Wie muss das Interface angepasst werden, um Kompilierfehler zu vermeiden?** Translator war ohne Modifier deklariert (interface Translator) und damit nur innerhalb des Packages control sichtbar. Der Client liegt im Package view und konnte das Interface deshalb nicht verwenden. Lösung: Das Interface wird als public interface Translator deklariert. Die Methode translateNumber ist in einem Interface implizit public, ebenso die Konstante version (public static final).

## Aufgabe 1.3 – Blackbox-Test

**Was ist der Vorteil einer separaten Test-Klasse?** - Produktivcode und Testcode sind sauber getrennt. Tests landen nicht in der ausgelieferten Software und blähen die Klassen nicht auf. - Die Tests prüfen die Klasse nur über ihre öffentliche Schnittstelle, also aus Sicht eines Nutzers (Blackbox). - Die Tests lassen sich unabhängig und automatisiert (z. B. mit JUnit oder im Build) beliebig oft wiederholen, etwa als Regressionstests nach Änderungen. - Die Übersicht und Wartbarkeit sind besser, und Tests und Code können getrennt weiterentwickelt werden.

**Was ist bei einem Blackbox-Test der Sinn von Äquivalenzklassen?** Man kann nicht alle möglichen Eingaben testen (bei int über 4 Mrd. Werte). Deshalb teilt man den Eingabebereich anhand der Spezifikation in Klassen ein, in denen sich das Testobjekt gleich verhält. Aus jeder Klasse genügt ein

Repräsentant: Funktioniert er, funktionieren vermutlich auch alle anderen Werte der Klasse. So erreicht man mit wenigen Testfällen eine systematische, hohe Abdeckung, sowohl für gültige (positive) als auch für ungültige (negative) Eingaben. Ergänzend testet man die Grenzwerte zwischen den Klassen, weil dort besonders häufig Fehler stecken (z. B. 0/1 und 100/101).

### **Warum ist ein Blackbox-Test mit JUnit auf der Klasse `Client` nicht unmittelbar durchführbar?**

Die Methode `display()` hat den Rückgabety `void` und gibt ihr Ergebnis nur über `System.out.println` auf der Konsole aus. Ein Testfall kann deshalb kein Ergebnis mit `assertEquals` mit einem Soll-Wert vergleichen. Außerdem ist `display()` `package-private` und vom Test-Package aus nicht aufrufbar. Zudem erzeugt der `Client` sein `Translator`-Objekt selbst über die `Factory`, sodass man im Test kein eigenes (Test-)Objekt einschleusen kann. Um ihn zu testen, müsste man die Konsolenausgabe umleiten oder den `Client` umbauen, z. B. so, dass er einen Rückgabewert liefert.

### **Testfälle und Äquivalenzklassen**

Siehe `Testfaelle_Uebung1.xlsx` bzw. `Testfaelle_Uebung1.pdf` sowie die Testklasse `test/se1/ws26/tests/uebung1/GermanTranslatorTest.java` (17 Testmethoden mit 21 Testfällen).